



Universidad de las Fuerzas Armadas – ESPE

**Carrera de Ingeniería de Software
Departamento de Ciencias de la Computación**

Análisis y Diseño de Software – NRC 30724

Tema: Patrones Comportamentales

**Command
Iterator
Mediator**

**Díaz Stiven
Leitón José
Manosalvas Gabriel**

Quito, 26 de mayo de 2026

PATRONES COMPORTAMENTALES

Los patrones de diseño de comportamiento se centran en los algoritmos y la asignación de responsabilidades entre objetos, describiendo de manera formal los esquemas de comunicación interactiva.

1. Patrón Mediator (Mediador)

Nombre del patrón: Mediator

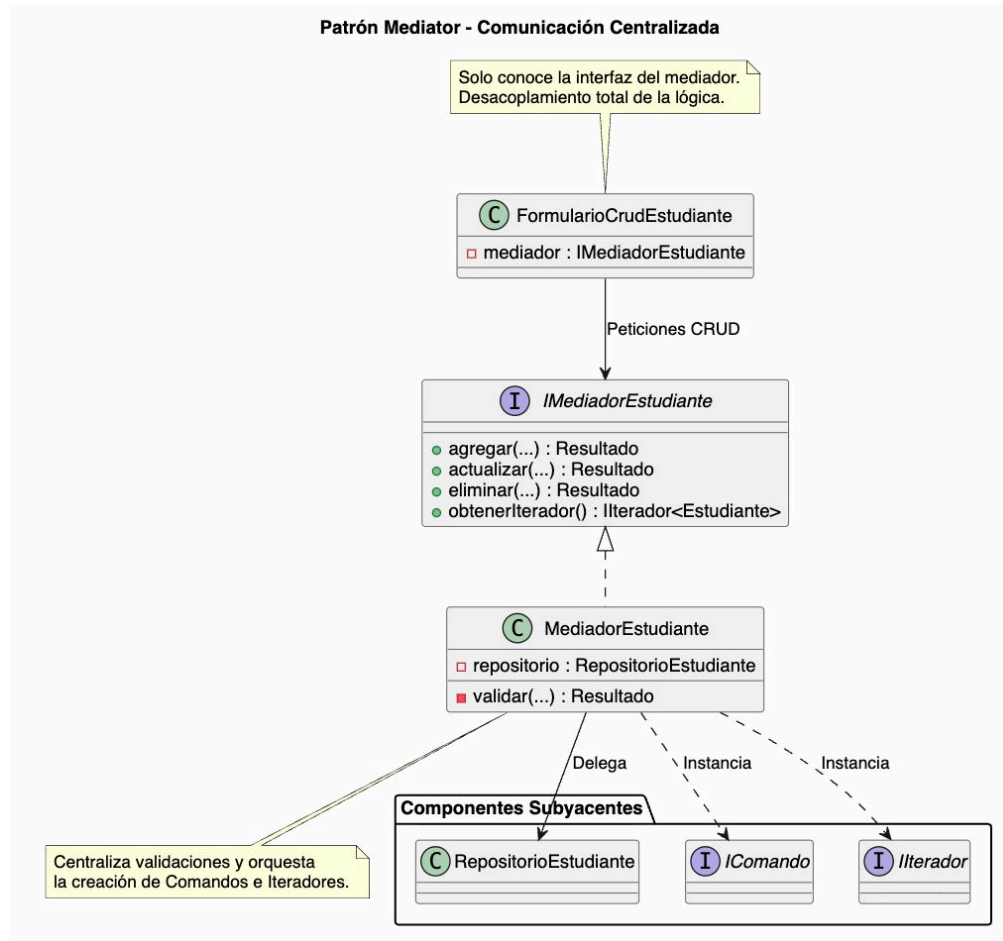
Descripción: Define un objeto centralizado que encapsula de manera estricta las interacciones lógicas y operativas de un conjunto de objetos cooperativos. Promueve un acoplamiento débil al restringir que las clases se comuniquen directamente entre sí, permitiendo modificar las dinámicas de interacción del subsistema de forma aislada.

Descripción del problema: A medida que un diseño de software evoluciona y se segmenta en clases especializadas, las comunicaciones cruzadas entre componentes tienden a incrementarse exponencialmente. Esta red caótica de dependencias mutuas directas (relaciones complejas de muchos a muchos) transforma el código en una estructura monolítica, frágil y rígida, donde la alteración menor de una clase impacta colateralmente en múltiples dependencias, anulando toda posibilidad de reutilización de componentes individuales.

Características:

1. **Mediador (Interfaz):** Modela el contrato de comunicación global y define los métodos conceptuales de despacho de señales utilizados por las clases asociadas para notificar eventos o transiciones de estado.
2. **Mediador Concreto:** Centraliza el núcleo de control coordinado del subsistema. Almacena referencias dirigidas a cada uno de los componentes de negocio (colegas) y arbitra las respuestas e interacciones requeridas ante las notificaciones entrantes.
3. **Componentes (Colegas):** Unidades lógicas independientes que ejecutan tareas funcionales del sistema. Mantienen una desconexión total sobre la existencia y estructura de los demás colegas, delegando cualquier requerimiento interactivo al objeto mediador central.

Patrón Mediator a un CRUD de Estudiantes



2. Patrón Command (Comando)

Nombre del patrón: Command

Descripción: Encapsula una petición o una acción como un objeto autónomo e independiente. Esto permite parametrizar a los clientes con diferentes solicitudes, estructurar colas o registros de operaciones y dar soporte nativo a operaciones que se pueden deshacer.

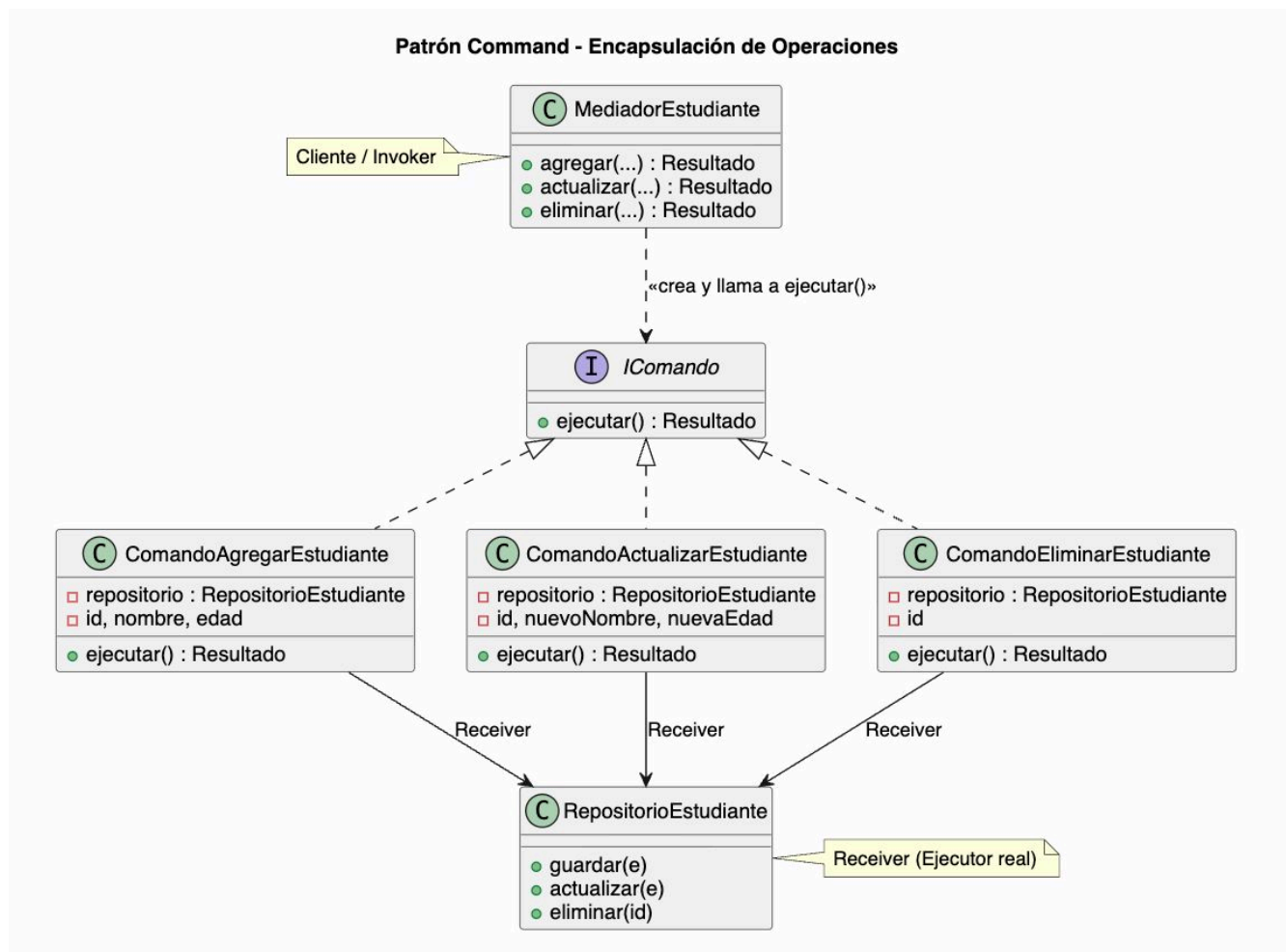
Descripción del problema: En el diseño de sistemas de software orientados a objetos, suele presentarse la necesidad de emitir solicitudes a componentes encargados de ejecutar tareas de negocio, pero sin acoplar directamente las clases emisoras con los detalles técnicos de los métodos o la identidad del receptor. Vincular de forma estricta al emisor con el receptor limita significativamente la flexibilidad arquitectónica, impidiendo la adición de operaciones en cola, auditorías o sistemas de historial reversibles.

Características:

- **Comando (Interfaz):** Define la abstracción unificada y el contrato formal para ejecutar una operación, implementando comúnmente un método genérico como Execute().
- **Comando Concreto:** Extiende la interfaz base de Comando, instanciando una vinculación específica entre un objeto receptor y una acción de negocio determinada. Al invocarse, delega el control llamando a las operaciones correspondientes del receptor.
- **Invocador (Invoker):** Componente responsable de iniciar la petición de control. Mantiene una referencia a la interfaz abstracta del Comando pero desconoce por completo la estructura interna y la identidad del receptor final.
- **Receptor (Receiver):** Clase de dominio que aloja la lógica de negocio pura. Posee los métodos y el conocimiento técnico detallado para ejecutar la acción solicitada de forma real.

Patrón Command (Comando): Puedes encapsular cada petición del CRUD (como "Crear Estudiante" o "Eliminar Estudiante") como un objeto autónomo e independiente. En la práctica, esto te permite parametrizar las solicitudes, estructurar colas de operaciones en el sistema y dar soporte nativo a operaciones reversibles, lo que te permitiría implementar un botón de "Deshacer", si se borra el registro de un estudiante accidentalmente.

Patrón Command un CRUD de Estudiantes



3. Patrón Iterator (Iterador)

Nombre del patrón: Iterator

Descripción: Proporciona un mecanismo formal para acceder secuencialmente a los elementos constituyentes de una estructura de datos o un objeto agregado complejo sin necesidad de exponer los detalles internos de su almacenamiento ni su representación física.

Descripción del problema: Las colecciones de datos complejas (tales como listas enlazadas, árboles binarios o grafos) requieren un canal de navegación para procesar sus elementos secuencialmente. Si los algoritmos de recorrido se implementan de forma directa dentro de la propia clase contenedor, esta se sobrecarga de responsabilidades arquitectónicas (violando el principio de responsabilidad única) y pierde la capacidad de permitir múltiples recorridos concurrentes, asíncronos e independientes sobre su propia estructura interna.

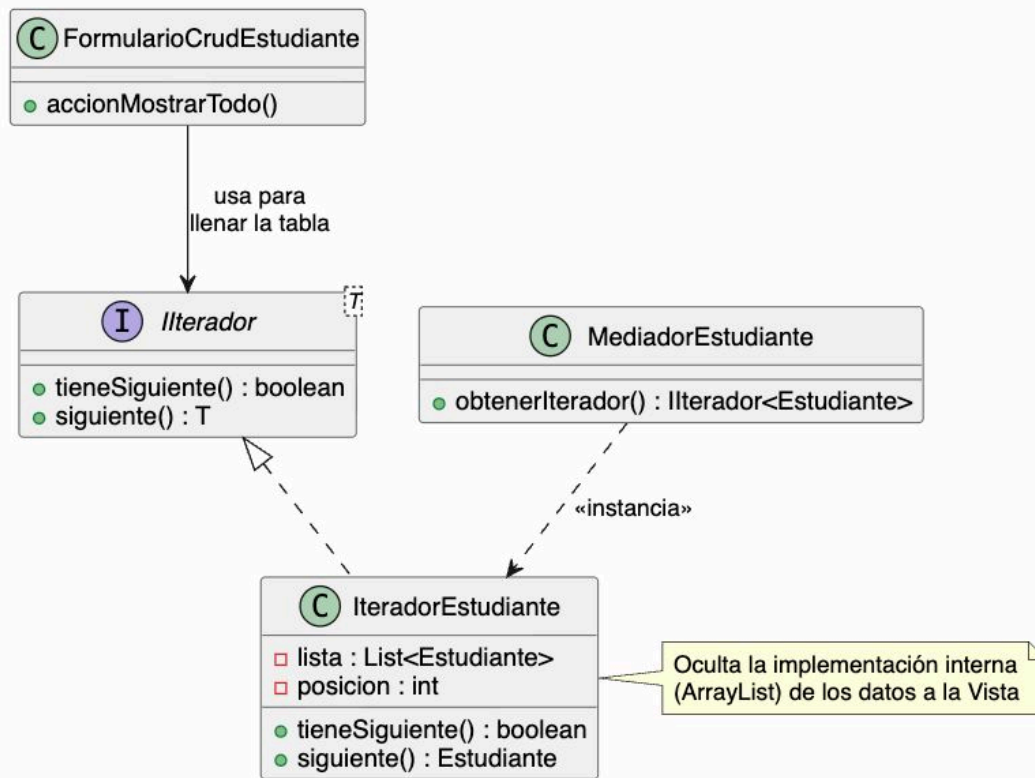
Características:

- **Iterador (Interfaz):** Modela las operaciones abstractas esenciales para controlar el flujo de navegación (métodos comunes para validar la existencia de elementos posteriores como `HasNext()` y para retornar la posición actual como `Next()`).
- **Iterador Concreto:** Implementa los algoritmos matemáticos y lógicos de recorrido específicos para una estructura de datos particular, manteniendo internamente el cursor de posición e historial de la iteración.
- **Agregado / Colección (Interfaz):** Declara la firma de un método de fábrica abstracta encargado de construir y retornar instancias de iteradores que sean formalmente compatibles con la colección de datos.
- **Agregado Concreto:** Implementa la interfaz del agregado, resguarda el almacenamiento físico de la información y genera el objeto iterador concreto correspondiente cuando un cliente lo demanda.

Patrón Iterator (Iterador): Al momento de "Leer" o listar el directorio de estudiantes, tu sistema podría requerir colecciones de datos complejas. Este patrón proporciona un mecanismo formal para acceder secuencialmente a los registros de los estudiantes sin exponer los detalles internos de su almacenamiento. De esta forma, extraes los algoritmos de navegación y evitas sobrecargar la clase contenedora con la lógica de cómo recorrer la lista.

Patrón Iterator un CRUD de Estudiantes

Patrón Iterator - Recorrido de Colecciones



Matriz Comparativa

El siguiente cuadro consolida la aplicabilidad conceptual de cada uno de los patrones estudiados para mitigar problemas específicos en la arquitectura de software:

Nombre del Patrón	Problema Asociado	Mecanismo Teórico
Command	Dependencia directa y rígida entre el objeto que solicita una acción y el objeto que la ejecuta.	Transforma la solicitud entera en un objeto autónomo con identidad propia.
Iterator	Exposición innecesaria del	Extrae los algoritmos de

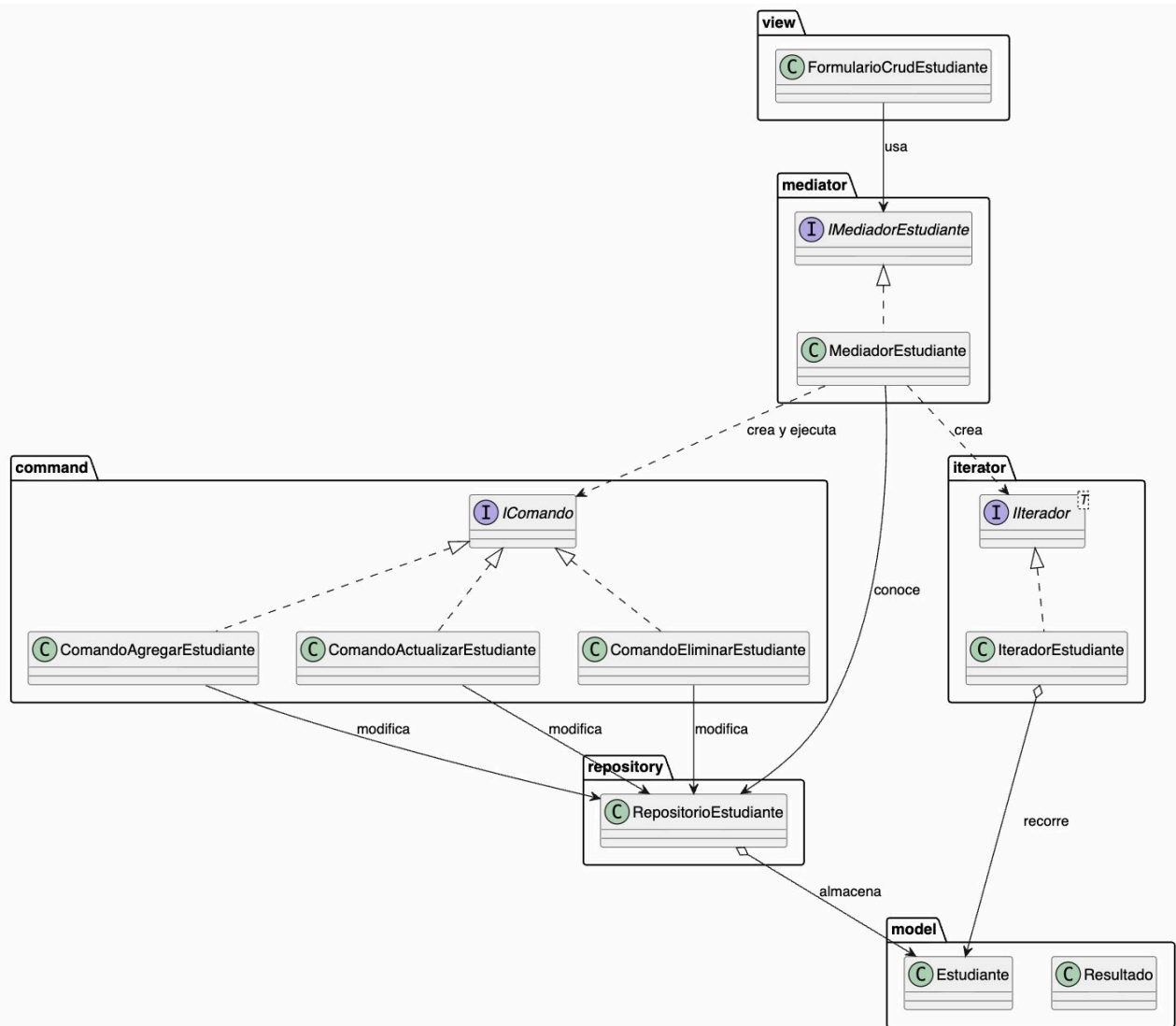
Nombre del Patrón	Problema Asociado	Mecanismo Teórico
	almacenamiento o de la complejidad interna de un contenedor de datos al recorrerlo.	navegación y control fuera del objeto que almacena los datos.
Mediator	Complejidad y acoplamiento difuso por dependencias cruzadas directas entre múltiples objetos cooperativos.	Interpone un componente central que intercepta e interviene en todas las comunicaciones del grupo.

Nombre del Patrón	Ventajas principales	Desventajas principales
Command (Comando)	• Desacoplamiento total: El emisor (UI) no sabe qué métodos usa el receptor. • Soporte nativo de Deshacer/Rehacer: Permite revertir operaciones fácilmente guardando el estado. • Extensibilidad: Es muy fácil añadir nuevos comandos sin alterar las clases existentes.	• Multiplicación de clases: El código puede volverse complejo al requerir una clase nueva por cada operación individual (Crear, Eliminar, etc.). • Mayor abstracción: Introduce capas intermedias que pueden dificultar el seguimiento simple del código.
Iterator (Iterador)	• Principio de Responsabilidad Única: Limpia la colección de datos de la lógica de cómo recorrerla. • Encapsulamiento: Oculta la estructura interna (si es una lista, un árbol o un grafo). • Recorridos concurrentes: Varios clientes pueden recorrer la misma lista de datos al mismo	• Sobrecarga para estructuras simples: Si tus datos se almacenan en una lista básica secuencial, aplicar el patrón puede ser innecesario e inflar el código. • Menor rendimiento: En algoritmos hipercríticos, la capa del objeto iterador puede ser ligeramente más lenta que un ciclo <code>for</code> directo.

	tiempo sin interferir entre sí.	
Mediator (Mediador)	• Reduce las conexiones “muchos a muchos” entre clases a relaciones simples “uno a muchos”. • Reutilización: Los componentes individuales (colegas) son autónomos y más fáciles de reutilizar. • Centralización: Simplifica la comprensión de cómo cooperan varios módulos del sistema.	• Riesgo de objeto Dios (God Object): Si no se diseña con cuidado, el Mediador concreto puede crecer desmesuradamente y volverse una clase monolítica difícil de mantener. • Punto único de fallo: Si el mediador falla, se interrumpe la comunicación de todo el subsistema.

Anexo:

Funcionamiento del CRUD Estudiantes, aplicado a los patrones.



Bibliografía

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Shvets, A. (s.f.). Command. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/command>
3. Shvets, A. (s.f.). Iterator. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/iterator>
4. Shvets, A. (s.f.). Mediator. Refactoring.Guru. <https://refactoring.guru/es/design-patterns/mediator>